

Richard van Zyl

Solutions Architect / Technical Lead — Backend & Data-Intensive Systems

Pretoria, Gauteng, South Africa · richardvzyl@gmail.com · linkedin.com/in/richardvzyl · github.com/RichardvZyl · richardvzyl.github.io

PROFESSIONAL SUMMARY

Solutions Architect and Technical Lead with **nine years** across C#/.NET and SQL Server — ~5 of them leading teams and ~3.5 as Solutions Architect — building high-throughput, multi-tenant financial systems where data integrity is non-negotiable. Deep specialist in backend engineering and relational data: execution-plan tuning, concurrency control, partitioning, and idempotent transaction design for ledgers operating across African markets. Requirements and solution design have run through the whole of that — specifying and modelling systems as a business analyst since 2017, pitched to whichever audience had to sign them off.

I own delivery end-to-end — from eliciting the requirement and modelling the solution, through concurrency and multi-tenancy strategy, to CI/CD and production telemetry — while leading teams and acting as the translation layer between engineering and the business. A problem solver at heart; languages and frameworks are simply tools. Regularly handed the challenges others could not solve, from turning around failed legacy migrations to designing the financial engine that replaced a legacy banking platform.

CORE COMPETENCIES

Requirements & Solution Design — Business (BRS), user (URS), functional (FRS) and technical (TRS) requirements specification; business- and problem-domain analysis; solution design and solution architecture; process and system modelling (UML / Enterprise Architect); stakeholder engagement and translation between business and engineering; audience-tailored technical writing.

Architecture & Design — Domain-Driven Design (DDD), CQRS, Event-Driven Architecture, Microservices, Multi-tenancy & Tenant Isolation, Idempotency / Exactly-Once Processing, RESTful API design, Clean Architecture.

Databases & Data — Microsoft SQL Server, Azure SQL, T-SQL, SQL Performance Tuning (execution plans, indexing, locking & isolation levels, deadlock resolution), Table Partitioning & Archiving, In-Memory OLTP, PostgreSQL, MySQL, Redis (distributed caching), MongoDB, Entity Framework.

Backend & Languages — C# (.NET Core, .NET 6+, ASP.NET Core, ASP.NET Web API), MediatR, FluentValidation, AutoMapper, Polly; SQL / T-SQL; TypeScript, JavaScript, Node.js.

Messaging & Integration — Azure Service Bus, RabbitMQ, API Gateway (Azure API Management), SignalR / WebSockets.

Cloud, DevOps & Observability — Microsoft Azure, Azure DevOps, **Azure DevOps YAML pipelines (pipeline-as-code)**, Docker on Azure hosts, AKS / Kubernetes, Azure CI/CD, GitHub, Trunk-based & GitFlow branching, Feature Flags. Observability: **New Relic** (release tracking wired through the DevOps pipeline, key-transaction instrumentation, alerting into MS Teams), OpenTelemetry, Grafana, Graylog over UDP, ActiveXperts, Azure Monitor / Application Insights / Log Analytics, Elastic Stack (ELK) / Serilog.

Security & Compliance — OAuth2 / OpenID Connect / JWT, Role-Based Access Control (RBAC), KYC / AML domains, SOX-aligned data segregation, data-protection-aware design (POPIA / GDPR).

AI & Agentic Engineering — Multi-model routing and evaluation (Claude, GitHub Copilot, Cursor, OpenAI Codex, Grok); agent orchestration with dependency-aware planning and budget governance; MCP server integration; automated standards review and analyzer-enforced quality gates; ADR-driven decision capture.

Testing & Tooling — xUnit, NUnit, Moq, Integration Testing (TestContainers); Enterprise Architect (UML), Swagger, Postman, SSRS; Angular, Angular Material; Agile, Scrum.

SELECTED ACHIEVEMENTS

- **Architected a multi-tenant, white-label financial engine** serving two operators (Betway and Jackpot City) from one configuration-driven codebase — 2 tenants, 25 brands, 150+ payment methods. Target isolation was database-per-brand; production constraint was one SQL Server with schema-per-brand and request-scoped contexts, because separate databases were priced out.
- **Sustained deposit volumes in excess of €10M on peak trading days**, absorbing both steady casino throughput and large spikes during live sporting events.
- **Eliminated double-spend race conditions** under heavy contention by designing idempotent attempt records keyed to a unique identifier, with rowversion-based optimistic concurrency on the ledger — exactly-once withdrawal processing under contention.
- **Migrated ~150 million records** across a high-throughput Saturday, at peak trading rather than a quiet maintenance window, with **no drop in the New Relic Apdex score**.
- **Solved problems others had abandoned**: completed a .NET 6 migration that had been attempted and rolled back by previous developers, and resolved a 2-year Angular Universal SEO blocker in one week.

PROFESSIONAL EXPERIENCE

Co-owner / Solutions Architect

Yuno Technologies · Digital consultancy · Apr 2026 – present

Half-owner of a South African digital consultancy, leading technical delivery — architecture, backend engineering and security design across client platforms.

- **Four-plane deployment topology** with one-way initiation between planes: a public edge behind CDN and WAF; an SPA plane running a backend-for-frontend that holds the session rather than the tokens; an application plane owning the signing key; and a response-only identity plane on Keycloak issuing OIDC / JWT. Each plane deploys independently with its own secrets, identity and network boundary.
- **Inter-plane trust model:** specified mutual TLS with SPIFFE/SPIRE workload identities at the service-mesh boundary, sequenced as a phase-two deliverable rather than forcing a Kubernetes-shaped stack onto the first release — with Keycloak service accounts as the documented interim, and the gap recorded as a critical open item rather than left implicit. (*Vite / React SPA, .NET services, PostgreSQL, Redis, RabbitMQ, outbox and dispatcher pattern, gRPC over HTTP/2 for service-to-service.*)
- **Isolation strategy for the ledger boundary:** moving the plane that carries the banking core off shared-kernel containers onto **KVM-backed microVM isolation** (RustVMM-based sandboxing — hardware-level separation at tens-of-milliseconds startup), so a container escape in an adjacent workload cannot reach the ledger. Currently under evaluation rather than in production.
- **Containerised platform across four environments** (development, staging, production and a legacy-migration path) — nginx edge, Django / Django REST Framework on PostgreSQL, Redis, Azure Functions, Azure Blob Storage and Communication Services, with Prometheus, Grafana and the Elastic Stack for observability, behind an isolated container network.
- Architecture decisions recorded as **ADRs with formal supersession**, so topology changes carry their own rationale rather than being reconstructed from commit history.

Solutions Architect / Technical Lead — Core Financial Systems

Raging River Trading (Pty) Ltd · Fintech / Gaming · Dec 2022 – Mar 2026

Led the architectural transformation of core financial systems, replacing a legacy banking platform with a modern, white-label engine handling high-volume transactions across Africa's largest gaming brands. Architecturally responsible for the processing flow, multi-tenancy strategy, and concurrency design, working in close partnership with the Enterprise Architect.

- **Multi-tenancy & data segregation:** Designed a three-level hierarchy — tenant, brand, payment method — in which **configuration inherits top-down and is overridable at any level, while financial data inherits nothing and is hard-segregated per brand**, each brand sitting independently in SOX scope. Brand does not reliably map to country (a cryptocurrency brand maps to none), so resolution ran per brand-and-payment-method with tenant defaults beneath rather than a country lookup. In production: 2 tenants, 25 brands, 150+ payment methods.
- **Schema-per-brand under a hard constraint:** Separate databases per brand were priced out and the data already sat in SQL Server, so 25 brands shared one database by schema separation — one EF Core code-first model defined once and deployed 25 times, giving every brand an identical shape and a single migration path instead of 25 divergent ones. Per-brand context instances with capped pools, bound to the request by an edge auth filter resolving API key to brand; contexts scoped per request and never shared, so isolation followed the request scope.
- **Ledger access boundary:** An orchestration API in front of a locked-down data-access API — only orchestration could mutate the ledger — with brand resolved from an API key at an edge filter, so no caller could reach another brand's data by construction.
- **Concurrency & ledger integrity:** Designed idempotent withdrawal processing under contention: each request creates an attempt record keyed to a unique identifier; on confirmation that identifier drives the ledger deduction, guarded by a rowversion (SQL timestamp) optimistic-concurrency check so updates only succeed if the row is unchanged since read — exactly-once behaviour for the debit. Supported both back-office-reviewed and automated "auto-cash-in" approval flows.
- **Request path (withdrawal):** REST initiation → input validation → configured strategy rules (FICA / playthrough / AML as required by brand and method, executed via an integration adapter) → attempt record keyed by a unique trace id (the idempotency key) → confirmation drives the ledger debit under **rowversion** → state change and outbox message commit in the same **SaveChangesAsync** → consumers drain; poison goes to a DLQ. A retry of the same trace id is the same attempt.
- **Work discovery & delivery:** Replaced chronological state-polling — contention by design. and liable to miss a row that commits after the scan has passed its timestamp — with a **transactional outbox committed inside SaveChangesAsync**, so a state change and the message announcing it land together and nothing scans business tables to find work. Drained to per-brand queues with dead-lettering, so segregation held to the messaging layer and no brand backlog could stall another.
- **Performance & storage strategy:** Applied in-memory (memory-optimised) tables with non-sequential keys on high-contention paths. Tuned fill factors separately to reduce last-page insert contention on hot indexes. Implemented tiered partitioning with progressive cold-archiving (detaching yearly partitions into separate databases on colder servers) to meet multi-year regulatory retention while keeping hot ledger tables performant.
- **Resilience:** Used exponential backoff with retries, circuit breakers, and dead-letter queues to prevent thundering-herd failures, on high-availability clusters with redundant storage.
- **Integration platform:** Worked on the platform where integrations are managed, routed and reconciled — abstracting 150+ money processors across online (callback/polling) and offline (e.g. USSD-reconciled) flows — and built integrations on it. Provider payloads were reshaped to the core contract by per-provider **IronPython** marshalling hosted in the .NET backend. **Rewrote the engine that moves each integration into its own instance**, so a single slow or chatty provider could no longer exhaust a shared thread pool or exhaust host sockets.
- **Large-scale migration:** Moved roughly **150 million records** across a high-throughput Saturday — peak trading rather than a quiet maintenance window — with **no drop in the New Relic Apdex score**.
- **Delivery & operations:** Practised trunk-based development with feature flags; defined build pipelines and shipped to production several times daily via automated CI/CD, with canary releases, health checks, and early-warning telemetry.

- **Stakeholders:** Acted as the bridge between engineering and the business (product owners, business analysts, marketing, client retention), translating feature needs into a sustainable architecture without sacrificing long-term integrity for short-term wins.

Segregation was enforced in application code — an auth filter and a resolver deciding which schema a request reached. It satisfies an auditor, but a single bug in that path has nothing beneath it; the barrier belongs in the engine, which on PostgreSQL is SET LOCAL search_path scoped to the transaction plus row-level security, where a missed predicate returns zero rows.

Role concluded via voluntary severance during a post-acquisition restructure.

Software Developer

MeterMo · Utilities / Automated Metering · Apr 2022 – Dec 2022

Platform modernisation and maintenance across an automated utility-metering estate — electricity, water and gas usage capture and reporting. Platform debt is a tax on every feature that comes after it; this cleared it.

- Migrated six projects from Team Foundation Server to Azure DevOps.
- Upgraded projects to current frameworks and package versions.
- Maintained two APIs, two cross-platform field applications (Xamarin, Cordova) and two websites (ASP.NET, Angular).

Software Developer (Specialist Problem-Solver)

Dotcom Software Solutions · Fintech consulting · Jan 2022 – Apr 2022

Specialist problem-solver for complex challenges other team members could not resolve, within the fintech consulting space for clients including Nedbank, Standard Bank, and PSG Wealth.

- **Legacy upgrade (Standard Bank – Schemes Bot):** Upgraded the Schemes (Stokvel / Chama) BotFramework backend from .NET Standard to .NET 6, resolving critical breaking changes and package dependencies — previously attempted and rolled back by others. Live and stable in production since February 2022. Also handled monetization design and payment validations. (*Azure SQL, BotFramework, C#.NET 6, MediatR, Q&A Maker.*)
- **Angular Universal SSR (PSG Wealth):** Implemented server-side rendering to resolve a 2-year SEO bottleneck in one week. (*C#.NET 6, Angular, Azure SQL, Docker, Node.js, Umbraco CMS, MediatR.*)

Senior Developer / Business Analyst / Team Lead

Payteq (Pty) Ltd · Fintech · Apr 2020 – Jan 2022

- **GoTrips (Team Lead | Full Rewrite):** Led the ground-up rewrite of a struggling platform, eliminating years of technical debt and introducing coding standards, design principles and performance optimisations. (*C#.NET Core API, PostgreSQL, Angular, Azure CI/CD, FluentValidation, Azure Functions.*)
- **Veriseal (Solo Migration):** Single-handedly migrated a critical compliance platform (KYC, AML, mass payments, bank-account verification) from legacy ASP.NET to .NET Core API with Angular 8, with meticulous attention to data integrity and security. (*.NET Core, SQL, SSRS, FluentValidation, AutoMapper.*)
- **GoBills (Team Lead):** Led development of a scalable business-management solution (inventory, POS, financials) with a sophisticated RBAC system governing permissions and client access. (*.NET Core API, Angular 8, Identity Framework, custom CMS.*)
- **Interchange IDE & Insight Server:** Go-to consultant for a highly scalable messaging-middleware integration platform with RBAC, on a SaaS basis.

Software Developer / Business Analyst

iPlan Global · Supply Chain / Manufacturing / Mining / Industrial · Oct 2017 – Apr 2020

Delivered tailored solutions across Supply Chain, Manufacturing, Mining and Industrial sectors as both developer and analyst, including solution-architecture design.

- **Consolidated Job Dashboard:** Automated a previously manual, error-prone supply-chain consolidation process. (*VBScript, SQL.*)
- **Espresso Quote Application:** Solo full-stack mobile quoting app with integrated email delivery and manager-approval workflows. (*C#.NET, Entity Framework, MVC, jQuery, AJAX, SQL.*)
- **ASP.NET Food Portal:** Distributed compliance/tracking system for farmers to log deliveries and track poison-testing results, enforcing user security across multiple locations. (*ASP.NET, C#.NET, XAML/MVVM.*)
- **Espresso RepCheckIn:** Rep-tracking and route-optimisation system with tiered priority visit lists. (*JavaScript, C#.NET, jQuery, MVC, AJAX.*)

Software Developer (C# .NET)

Novigo (Pty) Ltd · Recruitment · Apr 2017 – Oct 2017

Sole developer and IT administrator for a recruitment firm: designed, built and deployed a complete ERP from the ground up, including Linux server setup, domain hosting and general IT support. (*C#.NET WinForms, MySQL, Linux.*)

AI & MULTI-MODEL ENGINEERING

Multi-model evaluation practice. Runs five commercial assistants concurrently — Claude (primary), GitHub Copilot, Cursor, OpenAI Codex and Grok — as a deliberate comparison harness rather than a single-vendor commitment, with a considered position on where each is strongest and where consistency breaks down. Workflows are kept OpenAI API-compatible so any conformant provider is a drop-in.

Agent orchestration framework. Designed and built an end-to-end pipeline that decomposes a brief into a dependency-aware DAG of work items — each tagged with complexity, a preferred model tier and a token/tool-call budget — then executes them in parallel across specialist sub-agents under an overseer that resolves blockers, re-plans failed items and downgrades models on weighted cost/performance criteria. Architecture decisions are captured as ADRs *while the plan is formed*, not retrofitted afterwards.

Cost governance. Sub-agent execution is gated by user-defined session budgets with pre-flight estimates of token and tool-call spend and a live plan-versus-actual dashboard — the control layer that makes multi-agent work economically predictable rather than open-ended.

Automated quality gate ("strict mode"). A review workflow that grades output — from orchestration runs, peers or personal work — against versioned design-guideline references, then enforces it: all analyzers enabled, warnings as errors, a tiered diagnostic list and a self-extending recipe book of resolution strategies. Mechanical fixes route to cheaper models; only unknown diagnostics and public-API-affecting changes escalate. Includes custom Roslyn analyzers and a templated .editorconfig injected per project.

Two-axis automated code review. Separates *"does this follow our documented standards and ADRs?"* from *"does this do what the work item actually asked for?"*, running both as parallel sub-agents and reporting them side by side rather than collapsing them into a single verdict.

Reusable engineering tooling. Maintains a versioned library of skills, agents, commands and hooks with a two-layer publish/capture model, distributed across machines and packaged as a portable plugin. Conventions are published openly as engineering-standards (github.com/RichardvZyl/engineering-standards) — vendored into downstream repositories and kept current by an automated sync PR. Model context is extended through MCP server integration.

Frontier tooling evaluation. Runs a standing evaluation of emerging agent infrastructure beyond the mainstream assistants — agent frameworks (Hermes / Nous Research), model-routing layers (Omniroute, 9Router) and persistent memory and context systems (Perseus, Mnemonic AI) — tracking where the practical ceiling on context persistence, routing economics and agent autonomy is moving.

OPEN SOURCE

- **CombinatorialOptimiser** — github.com/RichardvZyl/CombinatorialOptimiser — permutation (TSP), subset selection (knapsack) and constraint assignment (graph colouring). Exact (Held–Karp, DP, branch-and-bound), construction, improvement (2-opt, Lin–Kernighan) and metaheuristics (simulated annealing, genetic algorithm, ILS). A solver registry recommends by instance size so $O(n!)$ is never the plan. Packable as NuGet; not published to nuget.org.
- **engineering-standards** — repository conventions used across delivery work, vendored downstream and kept current by an automated sync PR.
- **pseudo-random-guaranteed-unique** — T-SQL uniqueness without a random collision window.

NOTABLE CLIENTS ENGAGED (ACROSS CONSULTING ROLES)

Bankserv (now PayInc), Standard Bank, Nedbank, PSG Wealth, Libstar Holdings, Translution.

CERTIFICATIONS & EDUCATION

- **IKM C# Assessment:** 73rd percentile of all test takers (assessed twice, six months apart).
- **National Diploma — IT: Software Development:** Varsity College, 2014–2016.